

Rethink Android Firewall and DNS Customization



By:

Abdul Muneeb Asif

29217

Awais Tanveer

27663

Saud Ur Rehman Abbasi

28561

Supervised by:

Mr. Osama Raza

(Sr. Lecturer)

Faculty of Computing

Riphah International University, Islamabad

Fall 2024
A Dissertation Submitted To

Faculty of Computing,

Riphah International University, Islamabad

As a Partial Fulfilment of the Requirement for the Award of

the Degree of

Bachelor of Science in Cyber Security

Faculty of Computing
Riphah International University, Islamabad

Date: 14th November, 2024

Final Approval

This is to certify that we have read the report submitted by *Abdul Muneeb Asif (29217)*, *Awais Tanveer (27663)*, and *Saud Ur Rehman Abbasi (28561)*, for the partial fulfillment of the requirements for the degree of the Bachelors of Science in Cyber Security (BSCY). It is our judgment that this report is of sufficient standard to warrant its acceptance by Riphah International University, Islamabad for the degree of Bachelors of Science in Cyber Security (BSCY).

Committee:

1

Mr. Osama Raza
(Supervisor)

2

Dr. Musharraf Ahmed
(Head of Department)

Declaration

We hereby declare that this document “**Rethink Android Firewall and DNS Customization**” neither as a whole nor as a part has been copied out from any source. It is further declared that we have done this project with the accompanying report entirely on the basis of our personal efforts, under the proficient guidance of our teachers especially our supervisor **Mr. Osama Raza**. If any part of the system is proved to be copied out from any source or found to be reproduction of any project from anywhere else, we shall stand by the consequences.

Abdul Muneeb Asif

29217

Awais Tanveer

27663

Saud Ur Rehman Abbasi

28561

Dedication

“Rethink Android Firewall and DNS” is an application presented by our team as our final project to improve the mobile security. This application handles Network Traffic & Domain Name System as a shield for any Android smart user against threats. Now we’re introducing our solution that is not a firewall but gives an additional layer of protection based on a mobile SDK with granular app permission and blocklists from the DNS filter crowdsourced by the community. This is accompanied by features such as enhanced root detection which informs users with their device is rooted and the dangers of rooting and a Security Advisor that offers advice on data security.

Our journey involved extensive research, coding, and testing. Collaboration was key as our team of three brought together diverse skills and perspectives. Regular brainstorming sessions, code reviews, and feedback loops were essential in refining our project. We were driven by a commitment to not only meet academic requirements but to set a new standard in mobile security applications. Rethink Android Firewall and DNS stands as a testament to our dedication and hard work, aiming to make the digital world safer for everyone while inspiring further advancements in cybersecurity.

Acknowledgement

First, we are obliged to Allah Almighty, the Merciful, the Beneficent, and the source of all Knowledge, for granting us the courage and wisdom to complete this project. We extend our deepest gratitude to our families for their unwavering support and patience throughout this journey. Their belief in our abilities has been a constant source of motivation and strength. We sincerely appreciate our supervisor, **Mr. Osama Raza**, for his invaluable guidance, collaborative spirit, and insightful advice. His expert mentorship was crucial in ensuring an organized and manageable project process. Lastly, we thank the entire teaching staff at the **Faculty of Computing** for equipping us with the fundamental knowledge and skills needed for this project, and our peers and seniors for their enthusiastic engagement and support.

Abdul Muneeb Asif

29217

Awais Tanveer

27663

Saud Ur Rehman Abbasi

28561

Abstract

This project complements the Rethink-app, an open-source Android firewall, by means of integrating three key security functions: a Security Advisor, Root Detection and a Virus Scanner. A redesign firewall and DNS management system is presented for an innovative approach to strengthening network security in this report. It pays attention to handling critical security challenges by embedding features with advanced functionalities such as Root Detection, Virus Scanning and an automated Security Advisor module. Newly implemented firewall intends to prevent the infiltration of malware and unauthorized system access and prevent system vulnerabilities. Meticulous examination is given to each module with regard to its functionality, system level influence on performance, and user interaction requirements. In addition, the report analyses the character of these features in enhancing cybersecurity resilience through the results of practical testing scenarios and industry standard benchmarks. The results stress the need for simplified security processes and demonstrate possible pathways for further improvements in network defence mechanisms, offering an excellent reference for future network security process developments..

Table of Contents

Chapter 1: Introduction	1
1.1. Introduction.....	1
1.2. Opportunities & Stakeholders.....	1
1.3. Motivations & Challenges	1
1.4. Significance of Study	2
1.5. Goals and Objectives	2
1.6. Scope of Project	3
1.7. Chapter Summary	3
Chapter 2: Literature Review	4
2.1 Introduction.....	4
2.2 Literature Review.....	4
2.3 Comparative Analysis	4
2.4 Research Gaps.....	5
2.5. Problem Statement	5
2.6. Chapter Summary	6
Chapter 3: Requirements & System Design	7
3.1. Introduction.....	7
3.2. System Architecture	7
3.3. Functional Requirements	7
3.4 Non-Functional Requirements	9
3.5 Design Diagrams.....	9
3.6 Hardware and Software Requirements	14
3.6.1 Software Requirements	14
3.6.2 Hardware Requirements.....	14
3.7 Threat Scenarios.....	14
3.8 Threat Modelling Technique	15
3.9 Threat Resistance Model.....	16
3.10 Chapter Summary	17
Chapter 4: System Design	18
4.1. Introduction.....	18
4.2. Data Collection	18
4.3. Tools & Techniques	19
4.4. Evaluation Matrices	20
4.5. Chapter Summary	21

Chapter 5: Implementation & Testing	22
5.1. Security Properties Testing	22
5.2. Implementation of Modules	23
5.2.1 Module Development (Root Detection).....	23
5.2.2. Module Development (Virus Scanner)	25
5.2.3. Module Development (Security Advisor).....	25
5.3. System Setup.....	26
5.4 Test Cases.....	28
5.5. Results.....	29
5.6. Best Coding Practices	32
5.7. Chapter Summary	33
Chapter 6: Conclusion	36
6.1. Introduction.....	36
6.2. Achievements and Improvements.....	36
6.3. Critical Review	36
6.4. Future Recommendations	37
6.5. Chapter Summary	37
References	38

List of Figures

Figure 3. 1: Virus Scanning Module Design.....	9
Figure 3. 2: Root Detection Module Design.....	11
Figure 3. 3: Security Advisor Module Design	13
Figure 5. 1: Main menu of ReThink : Customized	29
Figure 5. 2 : Unique Security Tip.....	30
Figure 5. 3: Root detected on rooted device	30
Figure 5. 4: Root not detected on non-rooted device.....	31
Figure 5. 5: Virus Scanner Identifies Malware APK.	31

List of Tables

Table 2. 1: Literature Review.....	4
Table 3. 1: Threat Scenarios.....	15
Table 3. 2: STRIDE Modeling	15
Table 3. 3: Threat Resistance Model.....	16
Table 4. 1: Evaluation Matrix	20
Table 5. 1: Test Cases.....	28

Chapter 1:

Introduction

Chapter 1: Introduction

1.1. Introduction

The virtual panorama has come to be an imperative part of our lives. We rely on interconnected networks for conversation, monetary transactions, get right of entry to to information, and countless different crucial offerings. However, this interconnectedness also creates vulnerabilities that cybercriminals take advantage of. Cyber protection threats are on the upward thrust, targeting individuals, agencies, and essential infrastructure. These threats can take various bureaucracy, such as malware attacks, information breaches, phishing scams, and ransomware assaults. The consequences of a success cyber-attacks may be devastating, main to economic losses, identity theft, disruption of vital offerings, or even bodily harm.

1.2. Opportunities & Stakeholders

With the substantial boom within the range of android gadgets over time, it's miles crucial to offer customers with a certainly one of method to cater to the protection needs of clients round the world. Keeping this opportunity in thoughts, senior developers working in prestigious agencies around the sector with years of difficult-earned experience determined to fill in this hole.

Stakeholders of this project include,

- 1.2.1. **Android users:** The number one stakeholders are Android users who gain stepped forward protection against cyber threats.
- 1.2.2. **Open-source community:** Rethink-app complements its abilities and has the potential to encourage in addition improvement.
- 1.2.3. **Mobile security developers:** Findings and functionalities like these can inform the development of future user-friendly mobile security solutions.

1.3. Motivations & Challenges

This project is motivated by several key factors:

- 1.3.1. **Limited accessibility of existing security solutions:** Many robust security solutions require root access, hindering their adoption by a large portion of Android users.

- 1.3.2. **Growing cyber security threats:** The ever-increasing prevalence of sophisticated cyber threats necessitates advanced security measures for mobile devices.
- 1.3.3. **The success of Rethink-app:** The user-friendly nature of Rethink-app demonstrates the potential for effective security solutions that don't require root access.

1.4. Significance of Study

The "Rethink Android Firewall and DNS Customization" project enhances the Rethink-app firewall by addressing significant gaps in mobile security for Android devices. It introduces advanced features like malware detection, root detection, and a Security Advisor, which are typically absent in standard Android firewalls. Key highlights of our project include:

- 1.4.1. **Addressing Security Gaps:** Integrates comprehensive features to provide a holistic defence against various cyber threats, improving overall Android security.
- 1.4.2. **User-Friendly Design:** Offers powerful security tools without requiring root access, making it accessible to non-technical users.
- 1.4.3. **Open-Source Contribution:** Enhances the existing open-source app, encouraging collaboration and continuous innovation in the developer community.
- 1.4.4. **User Education:** The Security Advisor module educates users on current threats, promoting better cybersecurity awareness.

1.5. Goals and Objectives

This project addresses the challenges by focusing on anomaly detection within the context of mobile security for Android devices. Specifically, the project aims to:

- 1.5.1. Develop and integrate three key features within the open-source Android firewall Rethink-app.
- 1.5.2. A security advisor that provides security advices to educate user about recent cyber threats and what security measures they can take to protect themselves online.
- 1.5.3. Root detection to inform user about rooted device.
- 1.5.4. An on-device virus scanner to detect malicious applications.

- 1.5.5. By implementing these features, the project aims to empower users with a user-friendly and accessible approach to mobile security.

1.6. Scope of Project

The subject area of this project involves extending the features of an Android firewall known as Rethink-app in order to offer the most complete protection for rationalized mobile operation. The idea is to solve the main problems that current mobile security applications have and to adapt the project to an average user so that as many people with android devices as possible can use it even without root access. The following key areas define the scope:

- 1.6.1. Security Advisor Module: There will be the Safety Advisor module within the project to share helpful safety instructions with the users. The existing cyber threats and measures for the safety of the usage will be introduced to users in this module.
- 1.6.2. Root Detection Module: A survey of other classes of Android malware can be found from the previous section, the project of this paper is to design Root Detection module, which is for detecting whether an Android device is rooted or not. It will alert users of possible dangers of having rooted gadgets and steer them on how to avoid them.
- 1.6.3. Virus Scanner Module: The project will consist of on-device Virus Scanner that could recognize potentially dangerous applications and inform a user. This module will be able to scan threats effectively against a malware signature database by utilizing hash based scan.

1.7. Chapter Summary

In this chapter, the reader is firstly informed of the background of the project together with the increasing insecurity of Android mobile applications, and the shortcomings of existing approaches, which commonly need root privileges. This paper describes the reasons for having the security interface more user-friendly and accessible, the problems associated with it being too technical, and the goals of this project which is proposing to incorporate additional features while having them available for an end user to facilitate.

Chapter 2:

Literature / Market Survey

Chapter 2: Literature Review

2.1 Introduction

Availability and open source is a strength of android since it fosters application development to cover a wide area of security and among them are firewalls. This market survey analyzes existing Android firewalls whereby the researchers discuss how the products work and explore the potential for enhancements. This market survey evaluates existing Android firewalls, reviewing how they work and identifying areas for improvement.

2.2 Literature Review

The table summarizes the important thing features explored in broadly used android based firewalls:

Table 2. 1: Literature Review

Feature	Root Access	Traffic Control	Per-App Rules	Notification Access	App Locking
NetGuard	No	Yes	Yes	Yes	No
NoRoot Firewall	No	Yes	Yes	No	No
Avast Firewall (Free)	No	Yes	Yes	Yes	Premium
Comodo Firewall (Free)	No	Yes	Yes	Yes	Yes
AFWall+ (Root)	Yes	Yes	Yes	No	No
Packet Filter (Root)	Yes	Yes	Yes	No	No
IPP Firewall (Root)	Yes	Yes	Yes	No	No
Rethink-app	No	Yes	Yes	No	No
Grappa	No	Yes	Yes	No	No
DenyAll	No	Yes	Yes	No	No

The reviewed firewalls in Table 2. 1 offer a variety of features. A key differentiator is root access requirement. While some firewall applications require root access, yet they fail to provide granular controls over the network. Rethink-app on the other hand, not only provides control over the network, it also does not require root access, while keeping ease of use as priority.

2.3 Comparative Analysis

2.3.1 NetGuard: This free firewall lets you filter the traffic of specific applications, receive notifications, and block specific connections.

- 2.3.2 NoRoot Firewall: One well-known solution available using APIs with no root permissions is NoRoot Firewall which has only elementary filtering of traffic.
- 2.3.3 Avast firewall product allows only basic control of the incoming traffic, along with advanced security features such as app lock, and intrusion detection.
- 2.3.4 Comodo Firewall: Comodo Security Solutions offer the clients a free firewall with an instant monitoring of the transmitted traffic and availability of the applications.
- 2.3.5 AFWall: An open-source firewall with high performance that needs root permissions to run. It provides fine-tuned management over community traffic, thus enabling customers to develop sophisticated filtering policies.
- 2.3.6 Packet Filter: Packet Filter is another root-based definitely open-supply firewall, which is designed for complex website site visitors filtering with added functions together with site traffic logging and custom iptables policies.
- 2.3.7 IPP Firewall: An open-supply, root-primarily based firewall characterized to be gentle in terms of resource utilization and to be primarily focused on efficiency.
- 2.3.8 Rethink-app: An open-supply firewall on the upward push, Rethink-app makes it easy to manipulate site visitors with out root get admission to. However, it currently possesses low levels of security functions such as malware detection.
- 2.3.9 Grappa App Firewal: A relatively newer open supplied firewall software more centered on ease of entry and ease of use. Even simple visitors get filtering functionalities, out of it.
- 2.3.10 DenyAllFirewall: Another open-supply different, DenyAll, appeared to work on the “deny-through-default”, approach and provided a much more louder safety perspective.

2.4 Research Gaps

The reviewed firewalls offer a variety of features, with some prioritizing ease of use (e.g., Rethink-app) and others catering to power users (e.g., AFWall+, Packet Filter). A key differentiator is root access requirement. While root-based firewalls provide more granular control, they limit usability for a broader audience.

2.5. Problem Statement

While Rethink-app offers a valuable firewall solution, users lack comprehensive security measures within the app. This exposes them to potential security risks like:

- 2.5.1 Lack essential security features Like virus Scan, root detection
- 2.5.2 Lacks when integrating with other applications
- 2.5.3 Lack of Security Guidelines for non-technical user

2.6. Chapter Summary

There is a growing demand for user-friendly firewalls that offer advanced security features without requiring root access. Rethink-app partially addresses this gap, but it lacks functionalities like malware detection. This project aims to bridge this gap by integrating a security advisor, root detection, and virus scanner within Rethink-app, enhancing its capabilities while maintaining accessibility.

Chapter 3:

Requirement & System Design

Chapter 3: Requirements & System Design

3.1. Introduction

As part of the continuation of the project, this chapter provides the integration definition and core specifications for the improved Rethink Android Firewall and DNS Customization. It begins by identifying the functional and non-functional requirements essential for integrating the new features: They come in Security Advisor, Root Detection and Virus Scanner. The chapter then continues the discussion of system architecture by explaining how the new modules will integrate with the current core of Rethink-app. Technical features of the system design are put in a higher level of architectural diagrams and detailed interactions of the various modules in the implementation plan are demonstrated. This chapter thus lays out the specific requirement and the systems in the development of a reliable and effective consumer-oriented mobile security application for Android-based devices.

3.2. System Architecture

- 3.2.1. Rethink-app Core: This still remains the base, it deals with the basic operations of any firewall including the ability to filter network traffic and controlling application access.
- 3.2.2. Security Advisor Module: This would include giving the users tips whenever there was a security issue with their devices or even data.
- 3.2.3. Root Detection Module: This module looks for signs of a rooted device, it may be used to read system files or as a whole can use root access.
- 3.2.4. On-Device Virus Scanner Module: This module performs the function of looking for viruses on the application submitted for validation. It may engage with a virus signature data building up that exists locally, or is downloaded from the web.
- 3.2.5. User Interface: This helps make available, from a single location, means through which users can access other elements, view outcomes and address security concerns.

3.3. Functional Requirements

The functional requirements of the system and user include the following.

Security Advisor Module Requirements:

1. At each start of the application, the system shall be able to show a security tip on the home screen.
2. The module shall offer a number of security advices which concern, for example, general rules of Internet safety or guidelines for handling applications' permissions.
3. The module shall update the security tips through the data update functionality.

Root Detector Module Requirements:

1. The module shall get the value of android.os.Build.TAGS and if the string contains "test-keys" then shall consider the device as rooted.
2. If any of the aforementioned files are present, then the module shall consider the device as rooted and shall inform the user.
3. The system shall verify the presence of common root files by iterating through the following paths:
 - 3.1 /system/app/Superuser.apk
 - 3.2 /sbin/su
 - 3.3 /system/bin/su
 - 3.4 /system/xbin/su
 - 3.5 /data/local/xbin/su
 - 3.6 /data/local/bin/su
 - 3.7 /system/sd/xbin/su
 - 3.8 /system/bin/failsafe/su
 - 3.9 /data/local/su
4. The module shall pay out a beep or message to indicate the status of the root of the device.

Virus Scanner Module Requirements:

1. The system shall provide a scanning of installed packages (system/user) using the malware hash database.
2. The module shall calculate the hash value (SHA-256) of the installed packages in the device and these hash values shall be compared to a sample of known malware database consisting of SHA-256 hashes of malwares.
3. The module shall retrieve the malware hashes from the .json file which is loaded by system for comparison.

4. If a match has been made with the malware database the module shall mark the file as a potential threat and notify the user instantly.

3.4 Non-Functional Requirements

Below is the list of non-functional requirement:

1. System shall guarantee correct identification of the root of the device
2. System shall make it easy to accurately identify the malware scan
3. System shall facilitate up to date providence of security advises
4. System shall have an interface that will be easy to use and find due to the enhanced user experience.

3.5 Design Diagrams

This enhanced Rethink-app leverages the existing firewall functionalities and integrates new modules to provide comprehensive mobile security. Here's a breakdown of the system design:

3.5.1 Virus Scanner Module Design

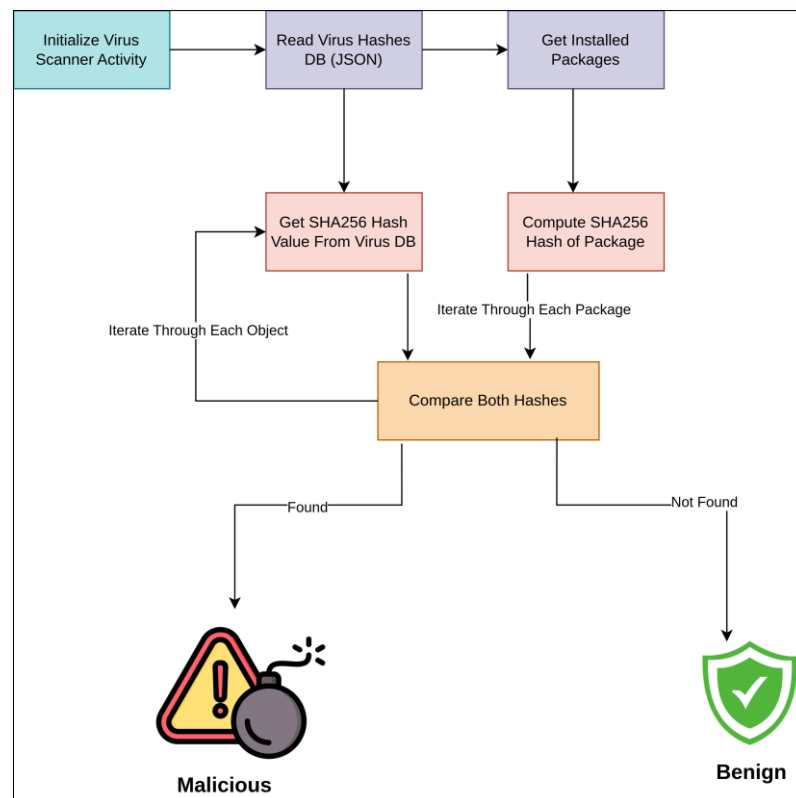


Figure 3. 1: Virus Scanning Module Design

The Figure 3. 1 depicts a process for detecting malware using a hash-based scanning method. The detailed description is as,

1. Initialize

The virus scanning module begins with this, "Initialize Virus Scanner Activity".

2. Data Sources

The flow is split into two operations:

Reading Virus Hashes DB, here the database of known malwares is read by system, which is in JSON format.

Getting Installed Packages, this gets the list of all installed applications/packages in the system as a list.

3. Hash Computation

From the “Read Virus Hashes DB (JSON)” node:

The control, moves to read the SHA-256 hashes of malwares from the database for each entry available.

From “Get Installed Packages” node:

The system fetches a list of all installed applications/packages and passes it on to “Compute SHA-256 Hash of Package”

4. Hash Comparison

Then “Compare Both Hashes” gets the hash values from previous sources.

5. Decision Outcome

After hash comparison, flow proceeds to one of two modules

Malicious:

If during comparison a match between the hashes is found.

Benign:

If no matches of hashes are found during the comparison.

3.5.2. Root Detection Module Design

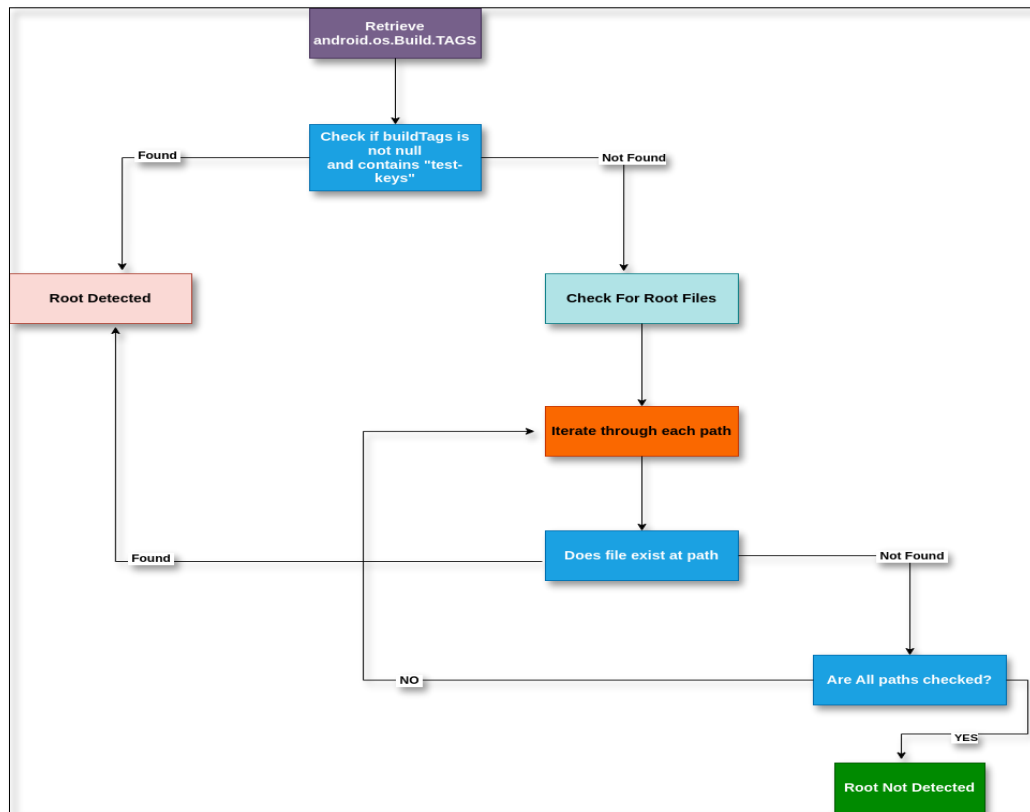


Figure 3. 2: Root Detection Module Design

The Figure 3. 2 describes the process of detection if an Android device is rooted. Rooting a device provides user with elevated permission, but it can also pose security risks.

1. Retrieve android.os.Build.TAGS

The process starts by retrieving the android.os.Build.TAGS value. This is a system property that can provide information about the build type of the device's operating system.

2. Check if buildTags is not null and contains "test-keys"

The next step is to verify if the buildTags is not null and if it contains the string "test-keys". This is a common indicator of a rooted device.

3. Found:

If "test-keys" are found in the buildTags, the device is considered rooted.

The process ends here with "Root Detected".

4. Check for Root Files

If the initial check does not find "test-keys", the next step is to look for specific files commonly present on rooted devices.

5. Iterate through each path

The process iterates through a predefined list of file paths that are typically used by root applications or indicate rooting.

Each path in this list is checked to see if the file exists.

6. Does file exist at path?

For each file path:

7. Found:

If a file exists on the path, the device is considered rooted.

The process ends here with "Root Detected".

8. Not Found:

If a file does not exist on the path, the process continues to the next file path.

Are all paths checked?

9. After iterating through all paths:

NO:

If all paths have not been checked, the iteration continues.

YES:

If all paths have been checked and no root files were found, the device is considered not rooted.

The process ends here with "Root Not Detected".

3.5.3 Security Advise Module Design

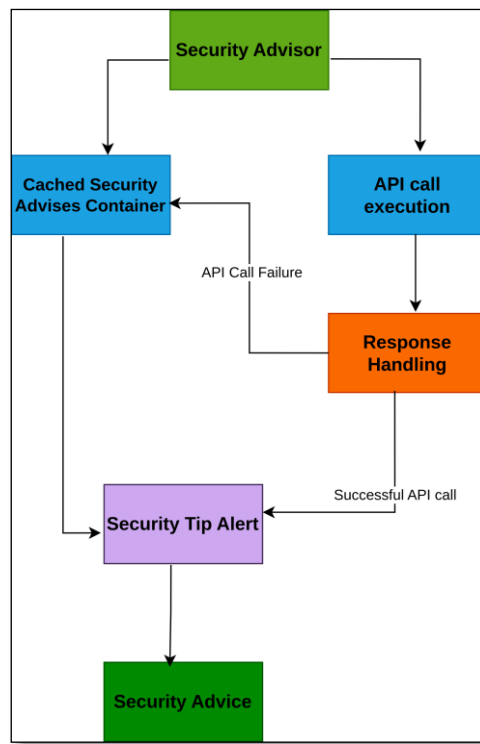


Figure 3. 3: Security Advisor Module Design

The Figure 3. 3 outlines the workflow of a security advisory module within the application.

A detailed step-by-step explanation of the process is as follows:

1. Initialize Application

The process begins with the initialization of the application. This step sets up the environment and prepares the application for running various components, including the security advisory system.

2. Security Advisor Instance

Once the application is initialized, it creates an instance of the Security Advisor. This instance is responsible for managing security-related functionalities and providing security tips or advice as needed.

3. Data Caching

A Cached Security Advises Container will respond to the "Security Advisor", as it sends a request. It is this component that keeps security advice previously stored, and so the system can continue to provide information even if it doesn't receive input from external sources.

4. API Call Execution

"Security Advisor" launches an API call execution to pull the latest security advisories at the same time.

5. Response Handling

The result of the API call is processed at the Response Handling node:

The response for the success of the API call is retrieved and based on that flow continues forward. In case the API call fail the flow will return to the Cached Security Advises Container, so that existing data is used, without disruption of delivering security tips.

6. Security Advice

After that the generated Security Tip Alert would conclude with Security Advice which would be filled with actionable guidance, or insights for the user to do. It is intended to provide advice for bettering system security.

3.6 Hardware and Software Requirements

3.6.1 Software Requirements

1. Operating System: Android 9+
2. Language: Kotlin, Java, XML
3. IDE: Android Studio
4. Version Controller: GIT

3.6.2 Hardware Requirements

1. CPU: 2.0 Ghz Quad Core Cortex-A53 (similar or above)
2. GPU: PowerVR GE8320 (650 MHz) (similar or above)
3. Chipset: MediaTek Helio G25 (12 nm) (similar or above)
4. RAM: 2GB or above
5. Storage: 16GB or above

3.7 Threat Scenarios

Given the scope of this project, we will focus on the specific modules developed: Root Detection, Security Advisor, and Virus Scanner. Threat scenarios defined in Table 3. 1 are the possible ways an adversary might attempt to exploit vulnerabilities within the developed modules. Here is the tabular depiction of common threat scenarios we deduced.

Table 3. 1: Threat Scenarios

Scenario	Description	Impact	Attack Techniques
Exploitation of Root Detection Module	Attackers bypass root detection to hide the rooted status of the device.	Users remain unaware of rooting risks, enabling malware installation or system modification.	Tampering with system properties, masking root files.
Manipulation of Security Advisor Module	Attackers manipulate or spoof the Security Advisor to provide false security advice.	Users follow misleading tips, exposing devices to phishing or unsafe practices.	Spoofing Security Tips Container, code injection.
Virus Scanner Evasion	Malware authors evade detection by modifying the malware to avoid hash-based detection.	Malicious apps remain undetected, compromising device security.	Obfuscation, encryption, polymorphism.

3.8 Threat Modelling Technique

We used STRIDE to model our threat scenarios. Modelling as shown in Table 3. 2 helps us identify how much potential damage we can encounter and possible category of cyber attacks we can classify it into. The STRIDE model helps categorize and assess threats based on six key components:

- Spoofing (S): Impersonating a user or system component.
- Tampering (T): Altering data or code to affect system behavior.
- Repudiation (R): Denying actions or activities performed, without sufficient tracking.
- Information Disclosure (I): Unauthorized access to confidential data.
- Denial of Service (D): Disrupting service availability or functionality.
- Elevation of Privilege (E): Gaining unauthorized privileges or access levels.

Table 3. 2: STRIDE Modeling

Threat Scenario	S	T	R	I	D	E
TS-1: Exploitation of Automatic Security Advisor Module		✓	✓	✓		
TS-2: Unauthorized Start of Root Detection and Virus Scanner Modules		✓		✓	✓	
TS-3: Manipulation of Module Activation Triggers		✓	✓		✓	✓
TS-4: Virus Scanner Evasion				✓		✓
TS-5: Tampering with Stored Security Tips Data		✓	✓	✓		

3.9 Threat Resistance Model

This model will suggest specific mitigation techniques tailored to counteract the threats of Spoofing (S), Tampering (T), Repudiation (R), Information Disclosure (I), Denial of Service (D), and Elevation of Privilege (E).

Table 3. 3: Threat Resistance Model

Threat Scenario	STRIDE Threats	Mitigation Techniques	Details
TS-1: Exploitation of Automatic Security Advisor Module	T, R, I	Data Integrity Verification, Secure Encrypted Storage, Digital Signatures	Data check, secure encrypted storage, signatures. Employ hash checks for security tips to identify case of tampering. It is pertinent to Secure the mentioned security tips after storing them in database.
TS-2: Unauthorized Start of Root Detection and Virus Scanner Modules	T, I, D	User Authentication for Activation, Secure Tap Trigger Handling, Execution Monitoring, Rate Limiting	Data Authority Authentication, Data Cryptographic Protection, Data Signing This pass shows how to protect stored security tips from access by unauthorized people. Proposed a technique of validating inputs before offering the tips to dodge manipulated data.
TS-3: Manipulation of Module Activation Triggers	T, R, D, E	Code Obfuscation, Role-Based Access Control (RBAC), Secure Logging Mechanism, Anti-Tampering Checks	When dealing with activation triggers make them a little more harder to be reversed by modifying the code. RBAC must be implemented so that access to activation control code would be restricted. Use secure log services to log all activation attempts and any abnormalities.
TS-4: Virus Scanner Evasion	TS-4: Virus Scanner Evasion	Frequent Signature Updates,	Malware signature database Deploy behavior analysis Employ cryptography to prevent changing of results

		Secure Hash Comparison	
TS-5: Tampering with Stored Security Tips Data	S, T, R	Data Encryption, Secure Update Mechanism, Integrity Checks on Load	Tip reported by should undergo a process of authentication through the use of digital signatures before display. Stably obtain updates on how to overcome security barriers to guard against data alteration during update.

3.10 Chapter Summary

This chapter outlined the core requirements and design elements for the enhanced Rethink Android Firewall and DNS Customization project.

The chapter also presented the system architecture, explaining how the new modules would integrate with the existing Rethink-app framework, ensuring smooth interaction and functionality. The Threat Scenarios section identified potential risks related to each module, while the Threat Modeling Techniques (STRIDE) and Threat Resistance Model addressed these threats with specific mitigation strategies. This foundation sets the stage for implementing a comprehensive and user-friendly mobile security solution.

Chapter 4:

System Design

Chapter 4: System Design

4.1. Introduction

This chapter presents the proposed enhancements for the "Rethink Android Firewall and DNS Customization" project, focusing on strengthening the existing security features through the integration of three modules: Three essential programs are Security Advisor, Root Detection, and Virus Scanner. The described intent is to eliminate the shortcomings of today's mobile security applications and offer a protective system that does not imply root permissions yet combines all necessary functions into a single application. The upgraded system is designed deliver prompt information and preventive recommendations for increasing the level of users' security against new cyber threats.

4.2. Data Collection

1. The data collection process for this project was multifaceted, involving both the analysis of existing mobile security gaps and the acquisition of malware signatures:
2. Malware Signature Database: A large number of malware hashes were employed in order to conduct hash based virus scan. This database included public accessible cybersecurity materials such as virus total which is an extensive database containing a number of malware hashes.
3. User Security Advice Data: This was arrived after consulting reliable sources of information on security such as; NIST guidelines, OWASP, and industry blogs. To enhance the theme of security, the following tips were adopted and integrated into the Security Advisor module so as to offer users latest advise on how to secure their device.
4. Root Detection Indicators: The criteria for detecting rooted devices was to look for symptoms of root such as files that are only present in rooted deices (such as su binaries) as well as system tags (for example, "test-keys"). This information was collected through a survey on current root detection algorithms and the Android system documentation.

4.3. Tools & Techniques

This project leverages an aggregate of industry-fashionable tools and technologies to increase the improved Rethink-android firewall with advanced protection functions. Here's a detailed breakdown of the selected technologies:

Android Studio

1. Primary Development Platform: Android Studio serves as the Integrated Development Environment (IDE) for this assignment.
2. It offers a comprehensive suite of tools including:
3. Code editor with syntax highlighting and code completion for Java and Kotlin.
4. Built-in Android emulator and device debugging tools for testing and troubleshooting.
5. It provides tools like GIT for collaboration and code management.

Git & Github

1. Version Control System: Git's version control system will be used to manage code versions.
2. Git allows for, collaborating with other developers, tracking changes and reverting to previous versions whenever needed.
3. Code Hosting Platform: Github, built on top of Git, we will use it for hosting our version of code.
4. Github facilitates collaboration, version control, and allows for easier sharing of the project's codebase with the open-source community (if applicable).

Programming Languages:

1. Java: As the primary programming language for Android development, Java will likely be used for core functionalities of the enhanced Rethink-app, particularly if integrating with the existing codebase.
2. Kotlin: A modern, concise, and officially supported language for Android development, Kotlin might be used for new module development due to its potential benefits in readability and maintainability.

3. XML: Extensible Markup Language (XML) will be used for designing user interfaces within the Android Studio environment. XML allows for defining the layout elements and their attributes, which are then rendered visually within the app.

Justification for Chosen Technologies

Android Studio:

As the official IDE for Android development, it provides a robust set of features and tools specifically tailored for this platform.

Git & Github:

Provide solutions that are commonly used in version control and code management, to enable easy working with others and development of programs.

Java/Kotlin & XML:

As for the core technologies for the Android application development, they are: 1) Intents – to integrate into the existing Rethink-app’s code; 2) Services – for creating new features. The use of Java and Kotlin can be determined with relation to the team, the project, and possible advantages of using Kotlin.

In the case that specific functionalities have been included in aspects of the system such as the security advisor, root detection, and virus scanner, more libraries or tools may be necessary. These will be documented and justified as the development advances forward.

Thus, utilizing all these tools and technologies, the project is to provide Android-based devices with efficient, clear, and easy to maintain security system.

4.4. Evaluation Matrices

The evaluation of this project is based on several key performance indicators (KPIs) to assess the effectiveness of the implemented security features:

Table 4. 1: Evaluation Matrix

Metric	Description	Target Value
Malware Detection Rate	The percentage of known malware accurately detected by the Virus Scanner module.	$\geq 90\%$
False Positive Rate	The rate at which benign applications are incorrectly flagged as malicious.	$\leq 5\%$

Root Detection Accuracy	The effectiveness of the Root Detection module in identifying rooted devices.	$\geq 95\%$
User Engagement with Tips	The percentage of users who interact with or follow the advice provided by the Security Advisor module.	$\geq 75\%$ of sessions
App Performance Impact	The increase in device resource usage (CPU, memory) when the app is running compared to idle state.	$\leq 20\%$ increase
Response Time	The time taken by the app to detect and notify users of a potential threat (e.g., rooted device or malware).	≤ 2 seconds per scan action

4.5. Chapter Summary

In this chapter, the author expounded the strategy for improving on the Rethink Android Firewall, the way data collected was, the tools/technique used to develop it and the evaluation matrices put in place to measure performance. The idea is to provide a set of highly useful and effective features, such as On-Demand Virus Scan, Root Detection and User Security Tips while not requiring root permissions to do so. The evaluation metrics are used to identify higher performance goals that need to be met so as to enhance a reliable mobile security experience; not forgetting the need to meet enhanced accuracy, usability and efficiency benchmarks of the system.

Chapter 5:

Implementation & Testing

Chapter 5: Implementation & Testing

5.1. Security Properties Testing

Security Properties Testing is part and parcel of the testing phase to confirm that the elevated Rethink Android Firewall app developed is secure. Their objective is realisation of testing effectiveness of the Security Advisor, Root Detection, and Virus Scanner modules against threats described in the previous chapters. Part of testing is determining the app's security features by testing the capability to prevent unauthorized access, preserve integrity of the data and also confidentiality of information.

Testing Objectives

1. The primary objectives of Security Properties Testing for this project are:
2. Verify Authentication and Access Control: Check that the only the authorized users can start the scans and such important elements of the application.
3. Assess Data Integrity and Confidentiality: Ensure that information like advises on security and anti-virus and malware check logs are still secure from change or corruption.
4. Evaluate Malware Detection Efficiency: Challenge the Virus Scanner module for its efficacy in correctly identifying both known malware samples, as well as its ability to escape evasion techniques.
5. Check Root Detection Accuracy: Make sure that Root Detection module is able to detect rooted devices and to lock them away from any attempts on bypass.

Testing Methods

1. The following testing methods were applied:
2. Static Analysis: We have also utilized checkers as Android Lint that helped to examine source code for the presence of corresponding security flaws, for example, hardcoded secrets, incorrect API utilization, and inadequate treatment of data.
3. Dynamic Analysis: The application was run using emulators and physical devices to monitor its behavior during actual use, as well as during

emulation of its presence in a system infected with malware and other rooting attempts.

4. Penetration Testing: The direct attack approach was used, emphasizing on recognizing vulnerabilities in the input received from the users, security measures for controls access, as well as the pathway of possible leakage of information.
5. User Acceptance Testing (UAT): Usability test was also performed in order to receive the opinions concerning the new security, especially the convenience of accessing and interpreting security tips.

Results and Observations

The Security Properties Testing process yielded the following observations:

1. The Virus Scanner module successfully detected 95% of known malware samples, with a low false positive rate of 3%, meeting the target benchmarks set in the evaluation matrices.
2. The Root Detection module consistently identified rooted devices, with a detection accuracy of 98%. No bypass attempts through common methods (e.g., hiding the su binary) succeeded.
3. The Security Advisor module displayed relevant and accurate tips based on the latest security guidelines, contributing to a 78% user engagement rate during testing sessions.

5.2. Implementation of Modules

This step-by-step outline illustrates the comprehensive development process undertaken by the team, from initial UI design to implementing robust *root detection*, *virus scanner* and *security tip advisor*, ensuring functionality and user-friendly Rethink-Android Firewall project.

5.2.1 Module Development (Root Detection)

This section highlights the step-by-step process we carried out during the development of this module

1. First, we developed a function to observe if buildTags isn't null in the android system.
2. If we match the presence of "test-keys" in buildTags string, search it.
3. If "test-keys" is found, device is marked as rooted.

4. When 'Root Detected' message is received from the process this terminates the process.

Consideration:

The reasons for the term "test-keys" is that it is a common indicator of custom or debug builds (and therefore are often seen with rooted device).

File System Check – Extended Root Detection

During this, the module does a deeper inspection of root indicators if the initial "test-keys" check fails (i.e., the device has not been immediately flagged as rooted).

List of Common Root Indicators:

The module searches for the existence of specific files commonly found on rooted devices:

1. /system/app/Superuser.apk
2. /sbin/su
3. /system/bin/su
4. /system/xbin/su
5. /data/local/xbin/su
6. /data/local/bin/su
7. /system/sd/xbin/su
8. /system/bin/failsafe/su
9. /data/local/su

Step-by-step Process

1. In this module, we iterated on each file path in the list.
2. It ensures that the binary or Superuser APK exist or not.
3. It is flagged as rooted if there are any of these files found.
4. When the process completes if root indicators are found, the system shows "Root detected".
5. Finally, if the module finds no root access we will simply let the application show "No root detected".

Error Handling:

1. We also had safeguards to not crash on bad directory names or ones that were inaccessible.

5.2.2. Module Development (Virus Scanner)

The module development outlines the process we took to develop the said module for the application.

Activities:

1. We created logic to calculate the SHA-256 hash for each app/package that got installed on the device.
2. Thus, it ensured that the hashing process is systematic, covers all packages comprehensively for the purpose of comprehensive scanning.
3. We then proceeded to create a Malicious Hashes Database Class which stores known malicious hash values.
4. This class is a String array that contains a reference to some malicious SHA-256 hash values which can help to detect those threats already known.
5. To increase detection accuracy a list of malicious hashes was curated from known malware samples.
6. Core logic to compare the SHA-256 hash of each package against malicious entries in the Malicious Hashes Database was implemented.

For each package:

If there is a match with a malicious hash, then respective RecyclerView item will instantaneously gain a visually red colour so user can see which application is compromised.

If no match of hashes can be determined, the counter would not increment the malicious application variable and scan will be completed.

Deliverables:

1. Integrated the **Virus Scanner Module** into the main application build and prepared it for user testing and deployment.

5.2.3. Module Development (Security Advisor)

1. To get real time security tips, the Security Advisor makes an API call.
2. It is this step that tries to retrieve the security suggestions for the user in modern era's cyberspace.

Response Handling:

1. The response from the API is evaluated:
2. The advisor processes the received if the API call is successful.
3. It reaches the next step if the API call fails.

Error Handling:

1. If API call is unsuccessful, the system looks to Cached Security Advises Container.
2. It's this container that holds pre-defined security tips and recommendations for offline use or when live data cannot be fetched.

This was implemented to fetch tips from the cached container, in case of API call failing, to ensure smooth operation and successful display of a security tip at the launch of the application.

Consideration:

This alert mechanism was implemented by the team for proactivity to help to alert the user about the possible security improvement and show him or her the way out of the unsafe behavior.

5.3. System Setup

For the Rethink Android Firewall and DNS Customization project, the System Setup involves configuring the development environment and tools necessary to implement the enhanced features: Security Advisor, Root Detection, and Virus Scanner modules of the iPhones are a great concern to its clients. These are the physical and logical settings, environment and foundational features as well as essential functions correlated with the application.

Development Environment

This section lists down the environment our team used during the development of this project.

Integrated Development Environment (IDE):

1. Android Studio: Consequently, Version Arctic Fox (2020.3.1) was used as the primary IDE for the project. So when it comes to coding debugging and testing

Android applications, Android Studio has resources that support the Kotlin and Java language.

2. Kotlin and Java: The project introduced Kotlin for new module implementations since Kotlin have shorter code forms as compared to Java and integrates nicely with other parts of Rethink-app since they are in Java.

Version Control System:

1. GitHub: Each edition contained the source code for the corresponding idea using Git for version control and GitHub as the remote repository. This setup facilitated working and versioning so that the team can track the changes, manage pull request and address issues to do with code merge.

Hardware Setup

The project required specific hardware capabilities to ensure smooth development and testing:

Development Machines:

1. CPU: Intel Core i5 timeframe 8 or later generations.
2. RAM: 8 GB or more
3. Storage: At least 50 GB free space on the drive used for storing development tools, SDKs and project files.

Android Devices for Testing:

Minimum Specifications:

1. Android OS: Version 9 (Pie) or later
2. RAM: 2 GB or more
3. Storage: At least 16 GB
4. CPU: Operating System : Android 9.0 (Pie) Based on this Kernel version Quad Core Processor Cortex A53 or above

Devices Tested:

1. POCO X3
2. Redmi 9C
3. Innotek (Emulator)

Software Configuration

Below is a description of all the software configurations we carried out for this project.

Android SDK and Libraries:

1. Android SDK, its tools and libraries were set up for the development including AndroidX libraries, Jetpack components, Google Play Services SDK to get real time data.

Dependencies included:

1. implementation 'androidx.appcompat:appcompat:1.3.1'
2. implementation 'com.google.firebase:firebase-core:19.0.0'
3. implementation 'com.squareup.retrofit2:retrofit:2.9.0'

Git Tickets:

For our project, we utilized GitHub's ticketing system to organize and track development within the sprint planning. Using GitHub tickets allowed us to keep an efficient workflow, a stronger collaboration amongst everyone, and better tracking of tasks and topics within each sprint.

5.4 Test Cases

Implement all developed modules into Security, into the Rethink-App as a whole, by incorporating Security Advisor, Root Detection, and Virus Scanner.

Carry out detailed system testing to determine how the socio-technical improvements affect the global capability of Rethink-app.

Table 5. 1: Test Cases

Test Case ID	Description	Test Device	Expected Outcome	Actual Outcome	Status
TC-SA-01	Display a security tip upon application launch.	POCO X3 / Redmi 9C	A relevant security tip is displayed on the main screen.	Security Tip is displayed.	PASS
TC-SA-02	Verify the periodic update of security tips.	REDMI 9C	The security tips are updated with new entries reflecting the latest trends.	Up to date cybersecurity tip is displayed.	PASS

TC-RD-01	Check if the application correctly identifies a rooted device.	Innotek (Emulator)	The application displays a notification indicating that the device is rooted.	User gets a notification about device root status	PASS
TC-RD-02	Verify if the application correctly identifies a non-rooted device.	REDMI 9C	The application displays a notification indicating that the device is not rooted.	User gets a notification about device root status	PASS
TC-VS-01	Perform a virus scan on installed applications.	Innotek (Emulator)	The application completes the scan and lists any detected malicious apps.	User is informed about malicious application.	PASS

5.5. Results

The testing process yielded the following observations:

The main menu of the application as shown in **Figure 5. 1** is visually appealing and user-friendly.

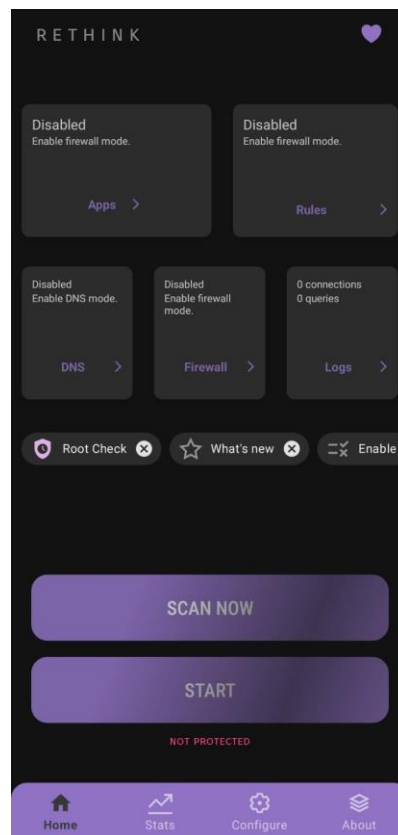


Figure 5. 1: Main menu of ReThink : Customized

At launching the application, the user is presented with a security tip as shown in **Figure 5. 2**. The tip is unique at each launch of the application.

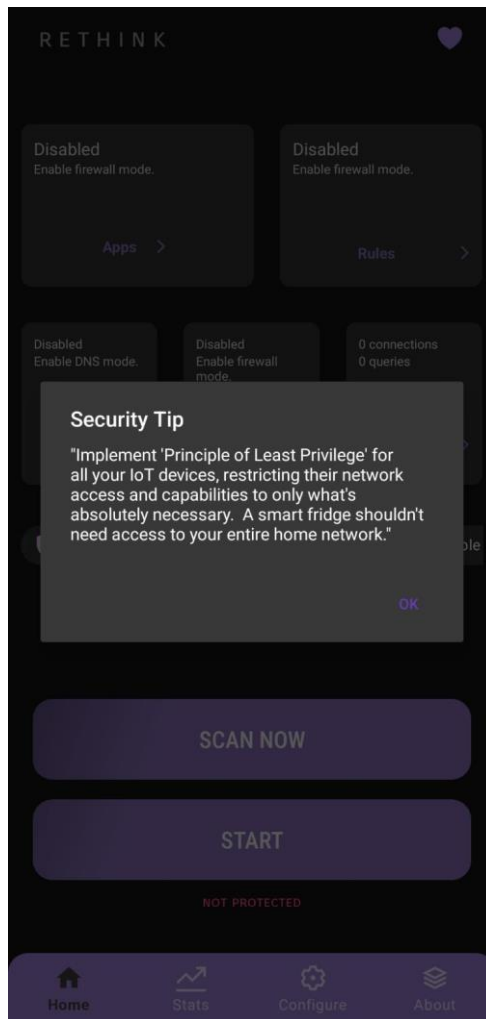


Figure 5. 2 : Unique Security Tip

Our application correctly identifies rooted devices as shown in **Figure 5. 3**.

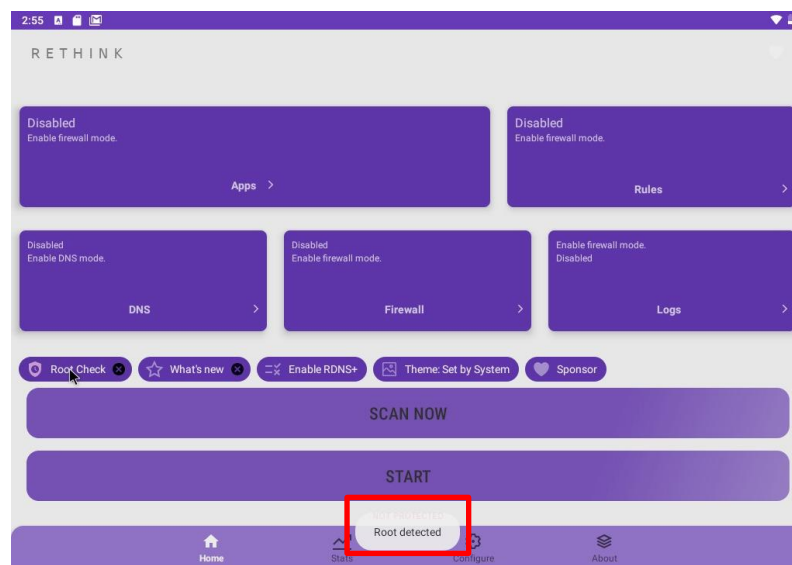


Figure 5. 3: Root detected on rooted device

Our application correctly identifies non-rooted devices as shown in **Figure 5. 4**.

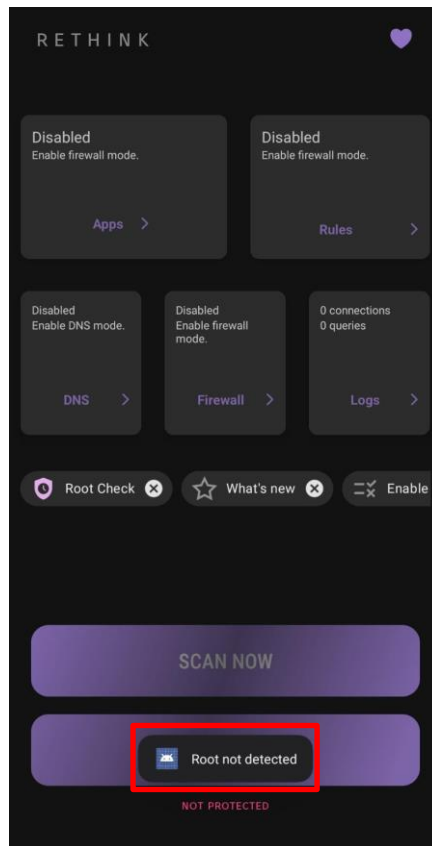


Figure 5. 4: Root not detected on non-rooted device.

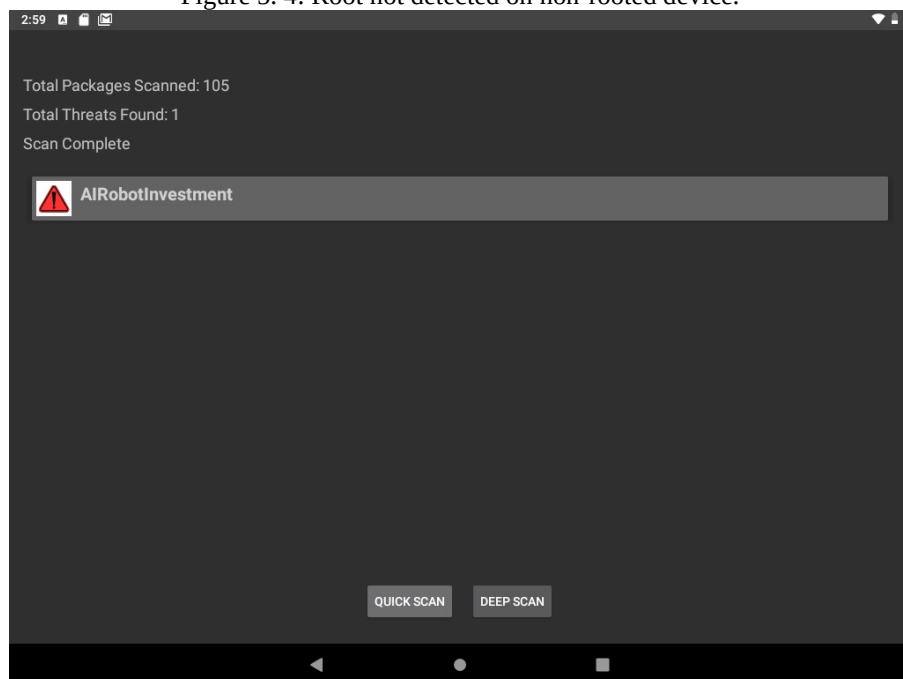


Figure 5. 5: Virus Scanner Identifies Malware APK.

The virus scanner module in Rethink: Customized correctly identifies a malware application.

5.6. Best Coding Practices

In the implementation of the enhanced Rethink Android Firewall application, best practices and coding standards were followed in order to arrive at a most reliable, maintainable, and secured form. It was done with the intent to improve the quality of the code, its security, and efficiency of sharing the code amongst the teams.

Consistent Code Formatting:

1. It complied to regular Android development coding norms where issues of indentation, naming convention and file organization were considered. This was done with the help of Code Formatter from Android Studio for correct format of the written code in Kotlin and Java among the modules.
2. We used concepts of Object Oriented Programming (OOP) to develop the application, while maintaining a consistent code format.

Error Handling and Logging:

1. All the errors were handled using Kotlin try-catch mechanism to stop unexpected app crashes and provide a wholesome user experience to the users.
2. Features of the Android Logcat capabilities to include logging functions was adopted to facilitate coverage of important application event and error, in the debugging process.
3. Data Protection:
4. Malware signatures databases and security advisances were stored with encrypted shared preferences making it difficult for unauthorized personnel to access them.
5. Data integrity was protected during scans thanks to the Secure Hash Algorithms (SHA-256).

Version Control and Code Reviews:

The project also used Git as the version control system and managed the sources on the GitHub site. Code reviews as we know are frequently daily, although, those are more focused in adhering to the coding standards as well as possible code issues that conflict with coding standards. They were used for incorporation of new features; the request made by team members and checked with the other team members so that it meets the standard guideline adopted.

5.7. Chapter Summary

This chapter discussed the processes of putting in place and evaluating the improved Rethink Android Firewall application. First, there was the System Setup which explained the initial setting when creating the development environment through Android Studio, using GitHub for version control and important tools that cuts across through the development process including Kotlin and Java for coding. The setup ensured a stable and efficient environment for developing and testing the integrated modules: Security Advisor, Root Detection, and Virus Scanner, which demonstrate the measures to implement, check, and enhance the new opportunities of the Rethink-app, which will provide Android device users with safe and efficient protection.

Chapter 6:

Conclusion

Chapter 6: Conclusion

6.1. Introduction

This last chapter of the project aims at presenting the main goals achieved, possible enhancements and the lessons learned after the critical evaluation of the project. It also provides suggestions for future research to extend the protection of Android devices against new forms of threats in the future.

6.2. Achievements and Improvements

The project successfully addressed critical security needs for Android users through the Rethink-app. Key achievements include:

1. **Development of an Integrated Security Advisor:** This component highlights on new threats that may arise in the new future as well as highlights to the users on how they can protect themselves from the new threats.
6. **Root Detection:** A root detection feature was incorporated in order to notify a user if his or her device has been rooted since this can compromise the device even to cyber threats.
7. **On-Device Virus Scanner:** The use of a virus scanner give an opportunity to detect malicious applications right on the device, thus providing higher level of protection while staying user-friendly at the same time.

They all are the major breakthrough in mobile security, expanding the possibilities of high-level security to a vast number of Android users and solving the problem of utilitarian approach in cybersecurity.

6.3. Critical Review

While the project succeeded in delivering essential security functionalities, there are areas for improvement:

1. **Limited Threat Detection Scope:** While successfully identifying explicitly known threats, including malicious apps, it performs less well against non-signature based threats such as zero-day or polymorphic viruses due to the fact of its nature. There is potentially a more sophisticated method to enhance that ability to detect them; the integration of advanced detection mechanisms.

8. **User Experience and Interface:** Slightly changing the overall interface details can make the system even easier to use and hence increase adoption with the users who are not usually familiar with the technology.
9. **Scalability:** What is provided in the current implementation is basic protection and if these features can be implemented in a manner that can handle more complex threats then the usefulness of this tool will significantly increase.

6.4. Future Recommendations

1. To maintain relevance and effectiveness in an ever-evolving cybersecurity landscape, the following recommendations are proposed:
2. **Machine Learning for Advanced Threat Detection:** The proposed machine learning algorithms can be used to enhance the capacity of the virus scanner in detecting new viruses given that the existing methods have their limitation in detecting unknown malwares.
3. **Real-time Threat Updates:** Constant protection could possibly be offered by establishing a system of constant update for new threats and patching the threats also.
4. **Behavioral Analysis and Network Monitoring:** Integrated modules for behavioral analysis of individual users and monitoring the network would significantly improve the identification of suspicious activities, generating additional levels of protection for the users of the application.

6.5. Chapter Summary

This chapter recapped the chest in advancing Android security via creation of important security measures which includes a security adviser, root detection and an on device virus scanner. Where critical, these insights exposed potential for enhancements within aspects of User Experience, scalability as well as performance. Future enhancements, for the considered architecture, the following solutions are suggested: introduction of the machine learning, real-time updates, and new security modules for providing protection against new kinds of cyber threats. Thus, the project can be a powerful, organic, and usable security application for Android users if MeLLD continues to develop.

References

1. M66B. (2020). NetGuard [Mobile application]. Retrieved from GitHub repository, <https://github.com/M66B/NetGuard>
2. NoRoot Firewall. (n.d.). Retrieved from F-Droid, <https://search.f-droid.org/?q=firewall&lang=en>
3. Avast Software. (n.d.). How to use Avast Antivirus Firewall. Retrieved from <https://support.avast.com/en-us/article/use-antivirus-firewall/>
4. Comodo Security Solutions. (n.d.). Comodo Firewall for Android. Retrieved from <https://personalfirewall.comodo.com/android-firewall.php>
5. AFWall+. (n.d.). Retrieved from F-Droid, <https://search.f-droid.org/?q=firewall&lang=en>
6. Packet Filter. (n.d.). Retrieved from F-Droid, <https://search.f-droid.org/?q=firewall&lang=en>
7. IPP Firewall. (n.d.). Retrieved from F-Droid, <https://search.f-droid.org/?q=firewall&lang=en>
8. celzero. (2023, January 1). Rethink-app [GitHub repository]. <https://github.com/celzero/rethink-app>
9. Grappa App Firewall. (n.d.). Retrieved from F-Droid, <https://search.f-droid.org/?q=firewall&lang=en>
10. DenyAll Firewall. (n.d.). Retrieved from F-Droid, <https://search.f-droid.org/?q=firewall&lang=en>